Importing Libraries and Preprocessing the data

Linear Regression

- It is supervised ML algorithm used to predict continous valued outputs(Scalars).
- In statistics, linear regression is a linear approach to modelling the relationship between a dependent variable and one or more independent variables. Let X be the independent variable and Y be the dependent variable. We will define a linear relationship between these two variables as follows: Y = mX + c
- we will use this equation to train our model with a given dataset and predict the value of Y for any given value of X.

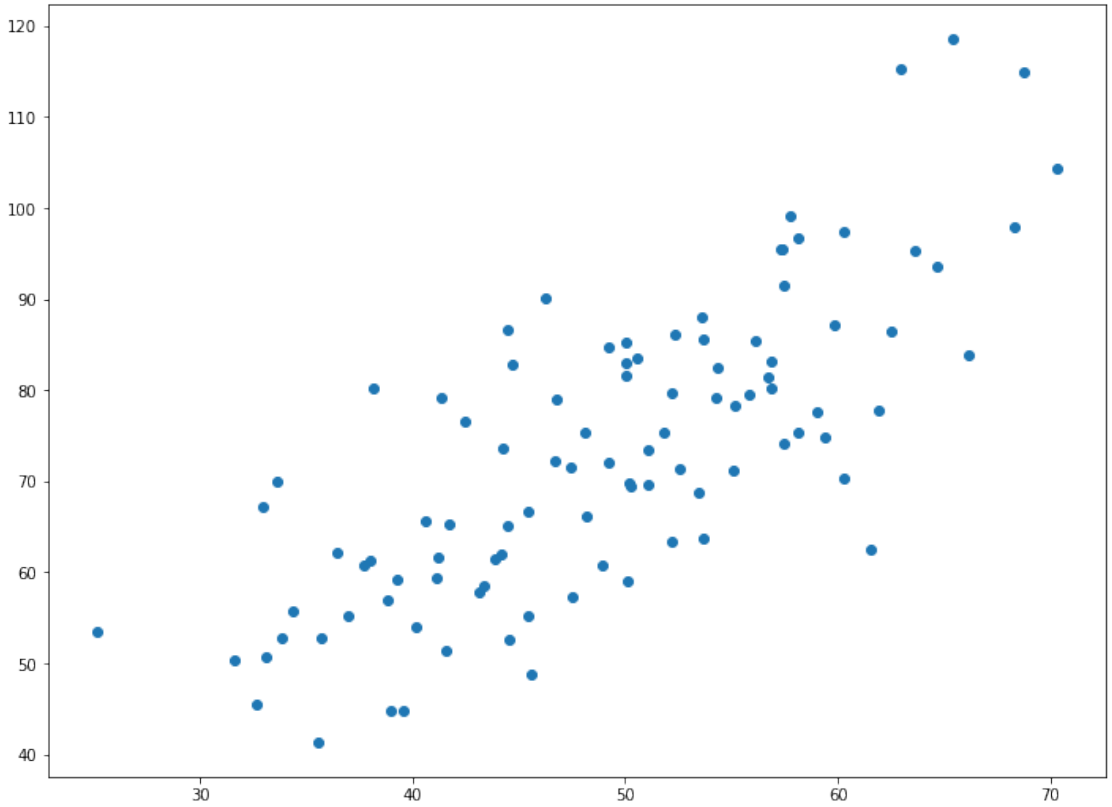
Our goal is to determine the value of m and c, such that the line corresponding to those values is the best fitting line or gives the minimum error.

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly
remount, call drive.mount("/content/drive", force_remount=True).
```

```
# Making the imports
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = (12.0, 9.0)

# Preprocessing Input data
data = pd.read_csv('/content/drive/MyDrive/Colab
Notebooks/prereq/assignment2_data.csv')
X = data.iloc[:, 0]
Y = data.iloc[:, 1]
plt.scatter(X, Y)
plt.show()
```



Cost Function

The loss is the error in our predicted m and c. Our goal is to minimize this error to obtain the most accurance value of m and c. We will use the Mean Squared Error function to calculate the loss. There are three steps in this function:

- Find the difference between the actual y and predicted y value(y = mx + c), for a given x.
- Square this difference.
- Find the mean of the squares for every value in X.

The average of loss over the entire dataset provides cost function

$$E = \frac{1}{n}\sum_{i=0}^{n}(y_i - (mx_i + c))^2$$

Gradient Descent

Gradient descent is an iterative optimization algorithm to find the minimum of a function. Here that function is our Loss Function.

GD steps:

- Init m=0 and c=0 and L be the learning rate(steps taken at each step).

- Calculate partial derivative of cost w.r.t m and c(Slope)

$$D_m = \frac{-2}{n} \sum_{i=0}^{n} x_i(y_i - \bar{y}_i)$$

$$D_c = \frac{-2}{n} \sum_{i=0}^{n} (y_i - \bar{y}_i)$$

- Update params m and c.

m = m - L * Dm

c = c - L * Dc

- We repeat this process untill our cost is minized and we get optimum value for m and c

Implementation

```
m = 0
c = 0

L = 0.0001  # The learning Rate
epochs = 1000  # The number of iterations to perform gradient descent

n = float(len(X)) # Number of elements in X

# Performing Gradient Descent
for i in range(epochs):
    Y_pred = m*X + c  # The current predicted value of Y
    D_m = (-2/n) * sum(X * (Y - Y_pred))  # Derivative wrt m
    D_c = (-2/n) * sum(Y - Y_pred)  # Derivative wrt c
    m = m - L * D_m  # Update m
    c = c - L * D_c  # Update c

print (m, c)
```

```
1.4796491688889395 0.10148121494753734
```

Learning Rate

- If the learning rate is large then we take large steps towards global minima and oscilates around global minima.
- If the learning rate is small then the cost updation will be small and takes more steps to converge to the global minima.

Prediction

```
# Making predictions
Y_pred = m*X + c
```

```
plt.scatter(X, Y)
plt.plot([min(X), max(X)], [min(Y_pred), max(Y_pred)], color='red') #
predicted
plt.show()
```

